```json
{
"name": "CB_GCal_Circuit_Breaker",
"nodes": [
{
"parameters": {
"path": "circuit-breaker/check",
"httpMethod": "POST",
"responseMode": "lastNode",
"options": {}
},
"name": "Webhook",
"type": "n8n-nodes-base.webhook",
"typeVersion": 2.1,
"position": [0, 0],
"webhookId": "circuit-breaker-check",
"id": "cb_webhook_001"
},
{
"parameters": {
"jsCode": "// Extract service info from request\nconst input = $input.first().json;\n\nreturn [{\n  json: {\n    service_id: input.service_id || 'google_calendar',\n    action: input.action || 'check', // 'check', 'record_success', 'record_failure'\n    error_message: input.error_message || null,\n    metadata: input.metadata || {}\n  }\n}];",
"options": {}
},
"name": "Parse Request",
"type": "n8n-nodes-base.code",
"typeVersion": 2,
"position": [250, 0],
"id": "cb_parse_002"
},
{
"parameters": {
"operation": "executeQuery",
"query": "SELECT * FROM circuit_breaker_state WHERE service_id = $1::text;",
"options": {
"queryReplacement": "={{ $json.service_id }}"
}
},
"name": "Get Circuit State",
"type": "n8n-nodes-base.postgres",
"typeVersion": 2.6,
"position": [500, 0],
"credentials": {
"postgres": {
"id": "SFNQsmuu4zirZAnP",
"name": "Postgres account"
}
},
"id": "cb_get_003"
},
{
"parameters": {
"conditions": {
"string": [
{
"value": "open",
```

```
"operation": "equals",
"property": "={{ $json.state }}"
}
]
}
},
"name": "Is Circuit Open?",
"type": "n8n-nodes-base.if",
"typeVersion": 2.3,
"position": [750, 0],
"id": "cb_if_open_004"
},
{
"parameters": {
"jsCode": "// Circuit is CLOSED - service is healthy\nconst state =
$input.first().json;\n\nreturn [{\n  json: {\n    allowed: true,\n    circuit_state:
'closed',\n    message: 'Service is healthy - requests allowed',\n    failure_count:
state.failure_count || 0,\n    service_id: state.service_id\n  }\n}];",
"options": {}
},
"name": "Return Closed State",
"type": "n8n-nodes-base.code",
"typeVersion": 2,
"position": [1000, -100],
"id": "cb_closed_005"
},
{
"parameters": {
"jsCode": "// Circuit is OPEN - check if timeout has passed\nconst state =
$input.first().json;\nconst now = new Date();\nconst openedAt = state.opened_at ? new
Date(state.opened_at) : null;\nconst timeoutSeconds = state.timeout_seconds || 300;\n\nlet
shouldTryHalfOpen = false;\nlet timeUntilRetry = 0;\n\nif (openedAt) {\n  const
elapsedSeconds = (now - openedAt) / 1000;\n  if (elapsedSeconds >= timeoutSeconds) {\n
shouldTryHalfOpen = true;\n  } else {\n    timeUntilRetry = Math.ceil(timeoutSeconds -
elapsedSeconds);\n  }\n}\n\nreturn [{\n  json: {\n    ...state,\n    should_try_half_open:
shouldTryHalfOpen,\n    time_until_retry_seconds: timeUntilRetry,\n    elapsed_since_open:
openedAt ? Math.floor((now - openedAt) / 1000) : null\n  }\n}];",
```

```
"options": {}
},
"name": "Check Timeout",
"type": "n8n-nodes-base.code",
"typeVersion": 2,
"position": [1000, 100],
"id": "cb_timeout_006"
},
{
"parameters": {
"conditions": {
"boolean": [
{
"value": true,
"operation": "equal",
"property": "={{ $json.should_try_half_open }}"
}
]
}
},
```

"name": "Timeout Passed?",
"type": "n8n-nodes-base.if",
"typeVersion": 2.3,
"position": [1250, 100],
"id": "cb_if_timeout_007"
},
{
"parameters": {
"operation": "executeQuery",
"query": "UPDATE circuit_breaker_state SET state = 'half-open', half_open_at = NOW(),
success_count = 0 WHERE service_id = $1::text RETURNING *;",
"options": {
"queryReplacement": "={{ $json.service_id }}"
}
},
"name": "Set Half-Open",
"type": "n8n-nodes-base.postgres",
"typeVersion": 2.6,
"position": [1500, 0],
"credentials": {
"postgres": {
"id": "SFNQsmuu4zirZAnP",
"name": "Postgres account"
}
},
"id": "cb_halfopen_008"
},
{
"parameters": {
"jsCode": "// Return circuit breaker status\nconst state = $input.first().json;\n\nreturn
[{\n json: {\n   allowed: state.state === 'half-open' || state.should_try_half_open,\n
circuit_state: state.state === 'half-open' ? 'half-open' : 'open',\n   message: state.state
=== 'half-open' \n   ? 'Circuit in half-open state - test request allowed' \n   :
`Circuit OPEN - retry in ${state.time_until_retry_seconds || 'unknown'}s`,\n   service_id:
state.service_id,\n   failure_count: state.failure_count || 0,\n   opened_at:
state.opened_at,\n   time_until_retry_seconds: state.time_until_retry_seconds || 0\n
}\n}];",
"options": {}
},
"name": "Return Open State",
"type": "n8n-nodes-base.code",
"typeVersion": 2,
"position": [1750, 100],
"id": "cb_return_open_009"
},
{
"parameters": {
"jsCode": "// Circuit is OPEN and timeout not passed\nconst state =
$input.first().json;\n\nreturn [{\n json: {\n   allowed: false,\n   circuit_state:
'open',\n   message: `Circuit breaker OPEN - service unavailable. Retry in
${state.time_until_retry_seconds || 'unknown'} seconds`,\n   service_id: state.service_id,\n
retry_after_seconds: state.time_until_retry_seconds || 300,\n   failure_count:
state.failure_count || 0\n }\n}];",
"options": {}
},
"name": "Return Blocked",
"type": "n8n-nodes-base.code",
"typeVersion": 2,

```
"position": [1500, 200],
"id": "cb_blocked_010"
},
{
"parameters": {
"jsCode": "// Standard Contract Output\nconst cbResult = $('Return Closed
State').first()?.json || \n                $('Return Open State').first()?.json || \n
$('Return Blocked').first()?.json;\n\nreturn [{\n  json: {\n    success: true,\n
error_code: null,\n    error_message: null,\n    data: cbResult,\n    _meta: {\n      source:
'CB_GCal_Circuit_BreakER',\n      timestamp: new Date().toISOString(),\n      version:
'1.0.0'\n    }\n  }\n}];",
"options": {}
},
"name": "Standard Contract Output",
"type": "n8n-nodes-base.code",
"typeVersion": 2,
"position": [2000, 0],
"id": "cb_sco_011"
},
{
```

```
"parameters": {
"path": "circuit-breaker/record",
"httpMethod": "POST",
"responseMode": "lastNode",
"options": {}
},
"name": "Record Result Webhook",
"type": "n8n-nodes-base.webhook",
"typeVersion": 2.1,
"position": [0, 400],
"webhookId": "circuit-breaker-record",
"id": "cb_record_webhook_012"
},
{
"parameters": {
"jsCode": "// Record success or failure\nconst input = $input.first().json;\nconst result =
input.success ? 'success' : 'failure';\n\nreturn [{\n  json: {\n    service_id:
input.service_id || 'google_calendar',\n    result: result,\n    error_message:
input.error_message || null,\n    metadata: input.metadata || {}\n  }\n}];",
"options": {}
},
"name": "Parse Record Request",
"type": "n8n-nodes-base.code",
"typeVersion": 2,
"position": [250, 400],
"id": "cb_parse_record_013"
},
{
"parameters": {
"operation": "executeQuery",
"query": "-- Record success/failure for circuit breaker\nWITH current_state AS (\n  SELECT *
FROM circuit_breaker_state WHERE service_id = $1::text\n),\nupdate_result AS (\n  UPDATE
circuit_breaker_state\n  SET \n    failure_count = CASE \n      WHEN $2::text = 'success'
THEN 0\n      ELSE failure_count + 1\n    END,\n    success_count = CASE \n      WHEN
$2::text = 'success' THEN \n        CASE WHEN state = 'half-open' THEN success_count + 1 ELSE
1 END\n      ELSE 0\n    END,\n    state = CASE \n      WHEN $2::text = 'failure' AND
failure_count + 1 >= failure_threshold THEN 'open'\n      WHEN $2::text = 'success' AND state
```

= 'half-open' AND success_count + 1 >= success_threshold THEN 'closed'\n   WHEN $2::text = 'success' AND state = 'open' THEN 'closed'\n   ELSE state\n   END,\n   opened_at = CASE \n   WHEN $2::text = 'failure' AND state = 'closed' AND failure_count + 1 >= failure_threshold THEN NOW()\n   ELSE opened_at\n   END,\n   last_failure_at = CASE WHEN $2::text = 'failure' THEN NOW() ELSE last_failure_at END,\n   last_success_at = CASE WHEN $2::text = 'success' THEN NOW() ELSE last_success_at END,\n   last_error_message = CASE WHEN $2::text = 'failure' THEN $3::text ELSE last_error_message END\n WHERE service_id = $1::text\n RETURNING *\n)\nSELECT * FROM update_result;",
"options": {
"queryReplacement": "={{ $json.service_id }}, {{ $json.result }}, {{ $json.error_message || '' }}"
}
},
"name": "Record Result",
"type": "n8n-nodes-base.postgres",
"typeVersion": 2.6,
"position": [500, 400],
"credentials": {
"postgres": {
"id": "SFNQsmuu4zirZAnP",
"name": "Postgres account"
}
},
"id": "cb_record_014"
},
{
"parameters": {
"jsCode": "// Return result of record operation\nconst state = $input.first().json;\n\nreturn [{\n json: {\n   success: true,\n   error_code: null,\n   error_message: null,\n   data: {\n   recorded: true,\n   service_id: state.service_id,\n   new_state: state.state,\n   failure_count: state.failure_count,\n   success_count: state.success_count,\n   message: `Recorded ${state.failure_count > 0 ? 'failure' : 'success'} for ${state.service_id}`\n   },\n   _meta: {\n   source: 'CB_GCal_Circuit_BreakER',\n   timestamp: new Date().toISOString(),\n   version: '1.0.0'\n   }\n }\n}];",
"options": {}
},
"name": "Record SCO",
"type": "n8n-nodes-base.code",
"typeVersion": 2,
"position": [750, 400],
"id": "cb_record_sco_015"
}
],
"connections": {
"Webhook": {
"main": [
[
{
"node": "Parse Request",
"type": "main",
"index": 0
}
]
]
},
"Parse Request": {
"main": [

```
[
{
"node": "Get Circuit State",
"type": "main",
"index": 0
}
]
]
},
"Get Circuit State": {
```

```
"main": [
[
{
"node": "Is Circuit Open?",
"type": "main",
"index": 0
}
]
]
},
"Is Circuit Open?": {
"main": [
[
{
"node": "Return Closed State",
"type": "main",
"index": 0
}
],
[
{
"node": "Check Timeout",
"type": "main",
"index": 0
}
]
]
},
"Check Timeout": {
"main": [
[
{
"node": "Timeout Passed?",
"type": "main",
"index": 0
}
]
]
},
"Timeout Passed?": {
"main": [
[
{
"node": "Set Half-Open",
"type": "main",
"index": 0
}
```

```
],
[
{
"node": "Return Blocked",
"type": "main",
"index": 0
}
]
]
},
"Set Half-Open": {
"main": [
[
{
"node": "Return Open State",
"type": "main",
"index": 0
}
]
]
},
"Return Closed State": {
"main": [
[
{
"node": "Standard Contract Output",
"type": "main",
"index": 0
}
]
]
},
"Return Open State": {
```

```
"main": [
[
{
"node": "Standard Contract Output",
"type": "main",
"index": 0
}
]
]
},
"Return Blocked": {
"main": [
[
{
"node": "Standard Contract Output",
"type": "main",
"index": 0
}
]
]
},
"Record Result Webhook": {
"main": [
[
```

{
"node": "Parse Record Request",
"type": "main",
"index": 0
}
]
]
},
"Parse Record Request": {
"main": [
[
{
"node": "Record Result",
"type": "main",
"index": 0
}
]
]
},
"Record Result": {
"main": [
[
{
"node": "Record SCO",
"type": "main",
"index": 0
}
]
]
}
},
"settings": {
"executionOrder": "v1"
},
"active": false,
"versionId": "cb-gcal-v1.0.0"
}